

LAPORAN PRAKTIKUM
PRAKTIKUM PENGEMBANGAN GAME
PERTEMUAN 4
INTERFACE, COMPONENT & COLLISION



Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
NIU	: 535493
Kelas	: PL5A1
Dosen Pengampu	: Yusron Fuadi, S.Sn., M.Sn.
Asisten	: Ezra Bariq Rizqullah / Ikhwan Hanif Firdaus

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Konsep Actor Component dalam Arsitektur Modular	4
2.2 Event Dispatcher dan Pemisahan Logika	4
2.3 Blueprint Interface untuk Komunikasi Antar-Aktor	5
2.4 Sistem Collision dan Deteksi Overlap	5
2.5 Variabel Instance Editable pada Desain Level	5
Hasil dan Pembahasan	7
3.1 Dokumentasi Praktikum	7
3.1.1 Pembuatan dan Konfigurasi Actor Component (BPC_Health) . . .	7
3.1.2 Pembuatan dan Implementasi Blueprint Interface (BPI_Overlap) . .	10
3.1.3 Blueprint Actor Collision dan Parameter	13
3.2 Hasil Akhir	15
Kesimpulan	18
Daftar Pustaka	19

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dasar dan implementasi Actor Component pada Unreal Engine 5.8 untuk membangun fitur game yang modular dan dapat digunakan kembali pada berbagai aktor [1, 2].
2. Mahasiswa mampu mengonfigurasi variabel kondisi kesehatan (CurrentHealth dan MaxHealth), merancang fungsi kalkulasi ModifyHealth, serta memanfaatkan Event Dispatcher (OnDeath) untuk mengabarkan perubahan status aktor secara terpisah tanpa keterikatan langsung [1, 3].
3. Mahasiswa mampu membuat dan menerapkan Blueprint Interface (BPI_Overlap) untuk mengirimkan pesan fungsi lintas aktor tanpa ketergantungan tipe data langsung (decoupling) [1, 4].
4. Mahasiswa mampu merancang aktor pemicu tabrakan (BP_Collision) berbasis Box Collision, menangani event overlap, dan mengatur variabel Instance Editable agar nilai damage dapat dimodifikasi secara langsung melalui viewport level [1, 5, 7].

Dasar Teori

2.1 Konsep Actor Component dalam Arsitektur Modular

Pada praktikum kali ini, pengembangan logika game beralih ke pendekatan modular menggunakan komponen. Di Unreal Engine, jika semua variabel dan fungsi ditulis langsung di dalam satu Blueprint Actor utama seperti karakter pemain, kode cepat membengkak dan sulit dipakai ulang untuk aktor lain. Memaksakan inheritance (pewarisan kelas) juga membuat struktur objek kaku [2].

Komponen hadir sebagai solusi: objek pembantu yang bisa ditempelkan ke dalam Blueprint Actor untuk memberi fungsi tertentu [1]. Unreal Engine membagi komponen menjadi dua jenis: Scene Component dan Actor Component. Scene Component punya wujud fisik dan koordinat transformasi (Location, Rotation, Scale) di dunia 3D, seperti Static Mesh atau Camera. Sebaliknya, Actor Component murni mengolah data dan logika tanpa butuh posisi di dunia [2]. Pada modul 4, sistem kesehatan karakter dibuat memakai Actor Component bernama BPC_Health [1]. Penamaan diawali prefix BPC_ (Blueprint Component) sesuai panduan penamaan aset resmi Epic Games [9].

2.2 Event Dispatcher dan Pemisahan Logika

Event Dispatcher bekerja mirip sistem radio pemancar atau pola publisher-subscriber di Blueprint [3]. Sebuah komponen dapat menyiarkan suatu kejadian ke lingkungan sekitar tanpa perlu tahu siapa yang mendengarkan atau aksi apa yang akan diambil pendengar tersebut.

Di dalam BPC_Health, event dispatcher bernama OnDeath dipanggil saat nilai CurrentHealth menyentuh angka nol atau kurang [1]. Komponen kesehatan cukup memanggil node Call OnDeath. Aktor pemilik seperti BP_ThirdPersonCharacter tinggal mengikatkan (binding) event tersebut ke aksi visualnya, misalnya mengecilkan skala karakter sampai nol [1]. Dengan pola ini, komponen kesehatan tidak terbebani urusan animasi atau efek visual kematian, sehingga logikanya tetap mandiri dan bersih.

2.3 Blueprint Interface untuk Komunikasi Antar-Aktor

Blueprint Interface adalah kumpulan deklarasi fungsi tanpa isi yang berfungsi sebagai kontrak komunikasi [4]. Di Unreal Engine, cara ini mengatasi kelemahan casting langsung (Cast to Class) yang biasanya membuat dua objek saling tergantung erat (tight coupling) dan membebani memori saat level dimuat [1, 10].

Modul mengibaratkan interface lewat rumus:

$$\text{Lingkaran.Luas()} \neq \text{Segitiga.Luas()} \quad (.1)$$

Kedua bentuk sama-sama menghitung luas, tapi rumusnya berbeda tergantung bentuk aslinya [1]. Pada praktikum ini, folder Interfaces dibuat di Content Drawer dan aset interface dinamai BPI_Overlap dengan fungsi TakeDamage bermasukan parameter float AmountDamage [1]. Aktor mana pun yang mengimplementasikan interface ini bisa menerima pesan damage tanpa aktor penyerang harus tahu siapa sasaran tembaknya [4].

2.4 Sistem Collision dan Deteksi Overlap

Collision di Unreal Engine bertugas mendeteksi kontak fisik antar-objek di level [5]. Interaksi tabrakan dibagi menjadi dua respons utama: saling memblokir (Block) atau saling menembus sambil memicu sinyal (Overlap) [6].

Untuk area pemicu (trigger volume), komponen Box Collision digunakan dengan opsi Generate Overlap Events aktif [1]. Saat karakter pemain berjalan melintasi kotak pemicu, event OnComponentBeginOverlap terpicu. Pin Other Actor disambungkan langsung ke pemanggilan fungsi pesan interface TakeDamage. Jika objek yang lewat tidak memiliki interface tersebut, Unreal Engine mengabaikan sinyal ini tanpa memicu error [4]. Secara default garis batas collision disembunyikan saat game di-Play lewat opsi Hidden in Game. Opsi ini dimatikan agar bentuk kotak collision terlihat langsung di viewport selama praktikum [1].

2.5 Variabel Instance Editable pada Desain Level

Variabel di Blueprint secara bawaan terkunci di dalam editor Blueprint itu sendiri [7]. Namun Unreal Engine menyediakan opsi Instance Editable (disimbolkan dengan ikon mata terbuka). Mengaktifkan opsi ini membuat variabel bisa diubah langsung lewat panel Details aktor yang ditaruh di level [1, 7].

Pada aktor BP_Collision, variabel float bernama **Damage** dijadikan Instance Editable. Hasilnya, desainer level bisa menaruh beberapa kotak pemicu di map dan memberi nilai damage yang berbeda-beda, misalnya kotak pertama bernilai 10 dan kotak kedua bernilai 30, tanpa

perlu membuat Blueprint baru [1]. Konsep ini sangat praktis untuk menyusun variasi rintangan di level.

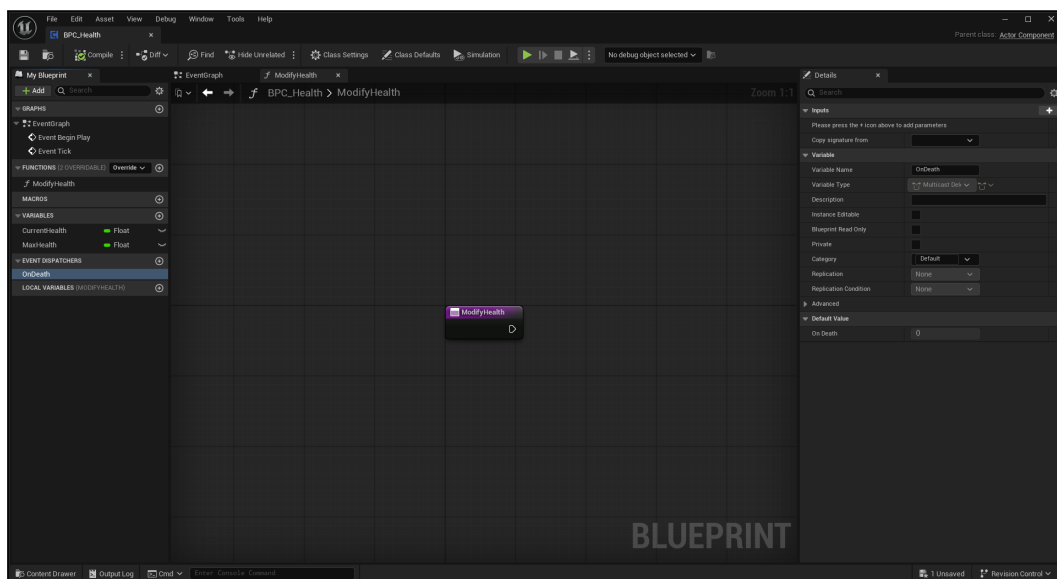
Hasil dan Pembahasan

Bab ini mendokumentasikan pelaksanaan praktikum Pertemuan 4 dengan topik Interface, Component, dan Collision. Praktikum saya kerjakan pada project Unreal Engine 5.8 HAFIDZ_535493_PPG dengan template Third Person. Seluruh tahapan disajikan secara berurutan sesuai alur modul praktikum beserta keterangan teknis pada setiap gambar.

3.1 Dokumentasi Praktikum

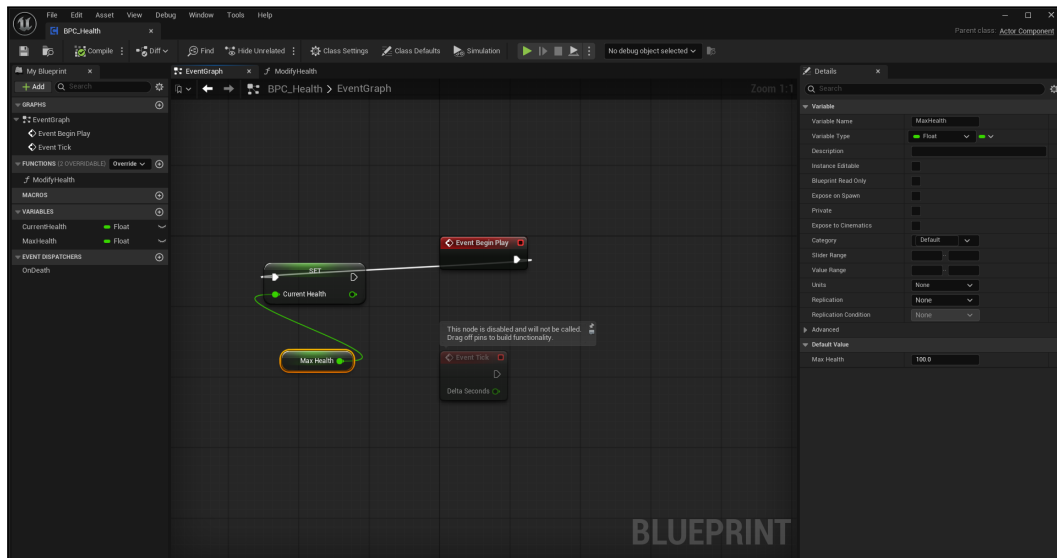
3.1.1 Pembuatan dan Konfigurasi Actor Component (BPC_Health)

1. **Inisiasi Actor Component BPC_Health.** Pada Content Drawer, tombol **Add** diklik untuk membuat aset Blueprint baru berkelas **Actor Component**. Komponen diberi nama **BPC_Health** dengan prefix standar. Di panel My Blueprint, dua variabel float dibuat yaitu **CurrentHealth** dan **MaxHealth**. Satu fungsi bernama **ModifyHealth** disiapkan, dan satu Event Dispatcher bernama **OnDeath** ditambahkan untuk menyiarkan status kematian ke aktor induk.



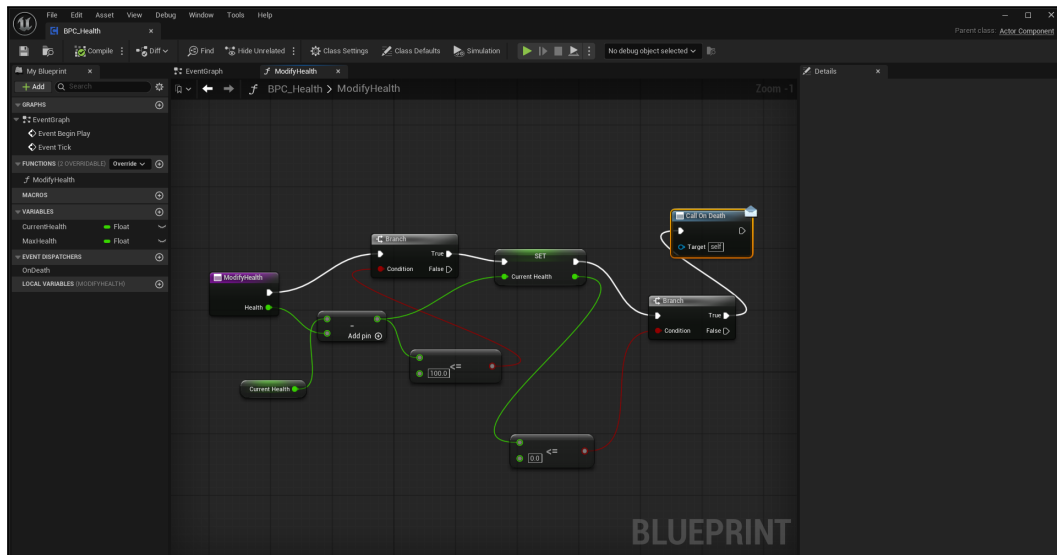
Gambar 1: Pembuatan Actor Component BPC_Health beserta konfigurasi variabel, fungsi, dan event dispatcher.

2. **Inisialisasi nilai kesehatan pada BeginPlay.** Di tab Event Graph milik BPC_Health, node **Event BeginPlay** dihubungkan ke node **SET CurrentHealth**. Nilai input SET disambungkan ke variabel **MaxHealth** supaya saat permainan dimulai, kesehatan karakter otomatis terisi penuh sesuai batas maksimal.



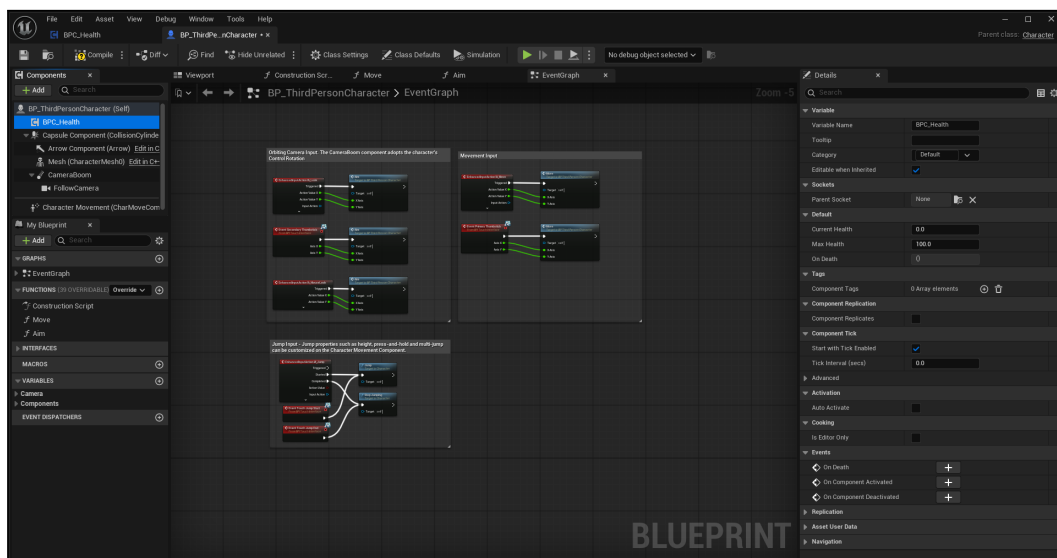
Gambar 2: Event BeginPlay pada BPC_Health yang mengisi CurrentHealth dari nilai Max-Health.

3. **Menyusun logika perhitungan fungsi ModifyHealth.** Fungsi **ModifyHealth** dibuka dan diberi parameter input float bernama **Health**. Logika graph disusun dengan mengurangi **CurrentHealth** dengan nilai input tersebut. Hasil pengurangan dicek menggunakan node Branch pertama, disimpan kembali ke variabel **CurrentHealth**, lalu diperiksa lagi dengan node Branch kedua untuk kondisi nilai kurang dari atau sama dengan nol (≤ 0.0). Jika True, alur memanggil node **Call OnDeath**.



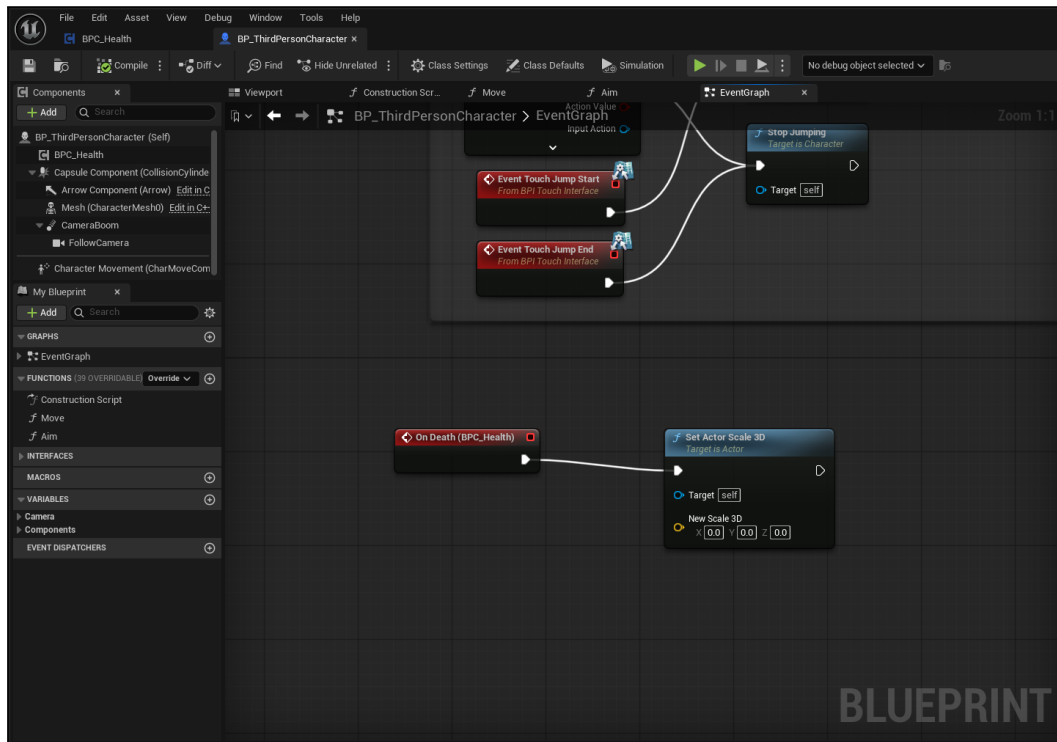
Gambar 3: Rangkaian graph fungsi ModifyHealth: kalkulasi pengurangan health dan pemanggilan Call OnDeath.

4. **Memasang BPC_Health ke BP_ThirdPersonCharacter.** Karakter BP_ThirdPersonCharacter dibuka pada Blueprint Editor. Komponen **BPC_Health** diseret dari Content Drawer ke hierarki Components milik karakter, sehingga karakter memiliki modul pencatat kesehatannya sendiri.



Gambar 4: Hierarki komponen BP_ThirdPersonCharacter setelah penambahan komponen BPC_Health.

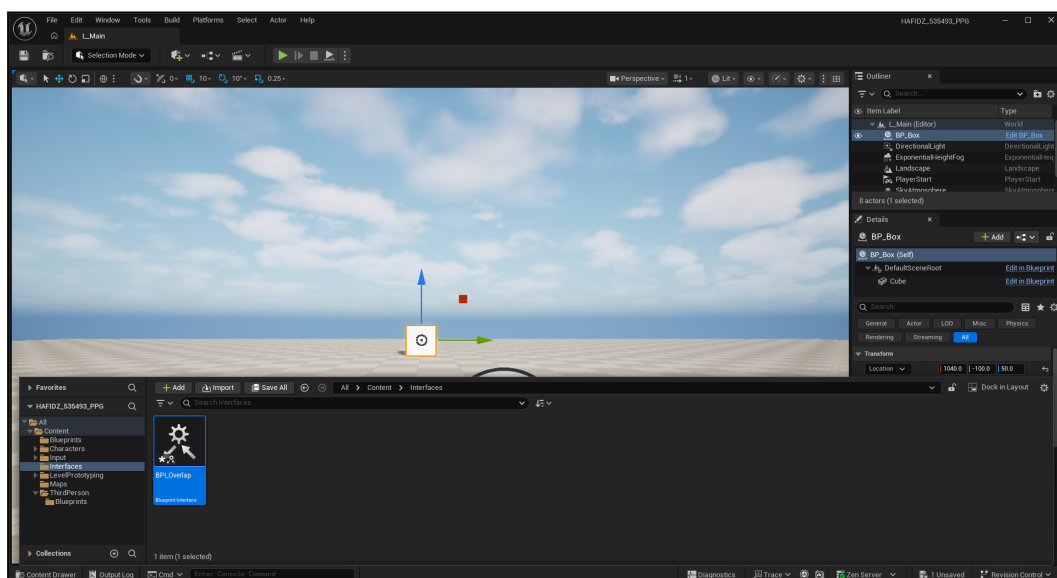
5. **Binding Event OnDeath pada karakter.** Pada panel Components karakter, komponen BPC_Health diklik kanan untuk menambahkan event **OnDeath**. Di Event Graph, node event ini dihubungkan ke node **Set Actor Scale 3D** dengan skala baru (0.0, 0.0, 0.0) dan target *self*, sehingga karakter menyusut dan menghilang saat kehabisan darah.



Gambar 5: Binding event OnDeath milik BPC_Health pada BP_ThirdPersonCharacter untuk mengubah skala karakter.

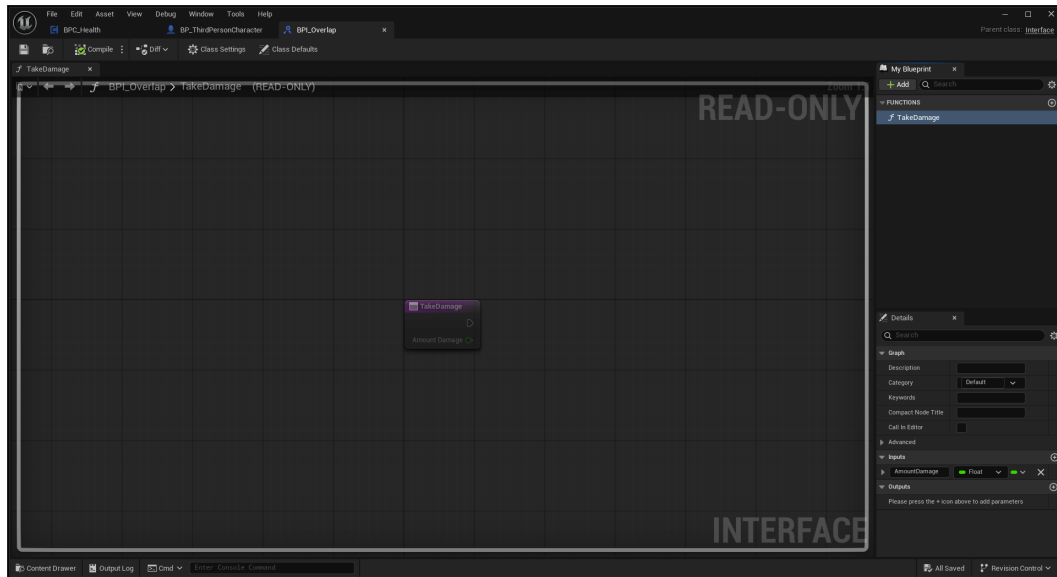
3.1.2 Pembuatan dan Implementasi Blueprint Interface (BPI_Overlap)

1. **Membuat folder Interfaces dan aset BPI_Overlap.** Folder baru bernama **Interfaces** dibuat pada direktori utama konten. Di dalam folder ini, aset baru ditambahkan lewat klik kanan menu **Blueprint** lalu memilih **Blueprint Interface**. File dinamai **BPI_Overlap** mengikuti aturan prefix standar.



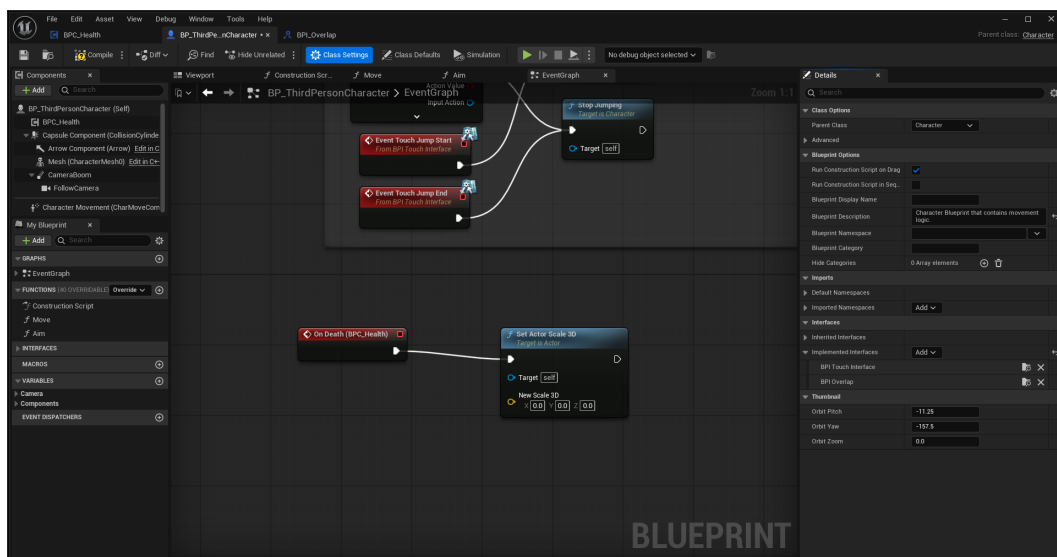
Gambar 6: Pembuatan folder Interfaces dan inisiasi aset Blueprint Interface BPI_Overlap.

2. **Mendefinisikan fungsi TakeDamage pada Interface.** Aset BPI_Overlap dibuka. Fungsi baru dibuat dan dinamai **TakeDamage**. Pada panel Details fungsi tersebut, sebuah parameter input bernama **AmountDamage** ditambahkan dengan tipe data Float.



Gambar 7: Definisi fungsi TakeDamage dengan parameter masukan AmountDamage bertipe Float pada BPI_Overlap.

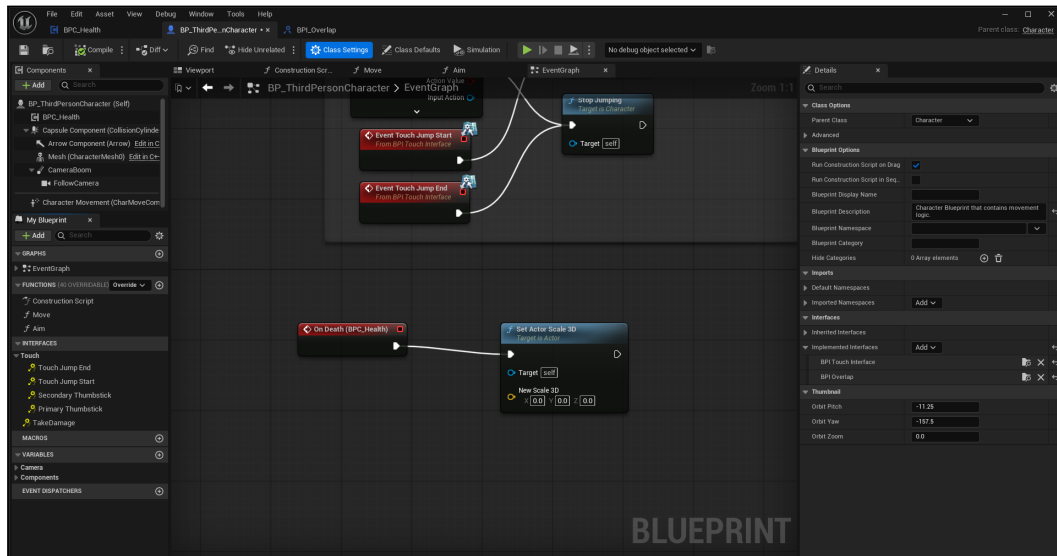
3. **Mengimplementasikan Interface pada BP_ThirdPersonCharacter.** Karakter BP_ThirdPersonCharacter dibuka kembali. Pada toolbar atas, menu **Class Settings** diklik. Pada panel Details bagian **Interfaces > Implemented Interfaces**, tombol **Add** ditekan untuk memilih dan menambahkan interface **BPI_Overlap**.



Gambar 8: Panel Class Settings BP_ThirdPersonCharacter saat menambahkan BPI_Overlap ke daftar antarmuka.

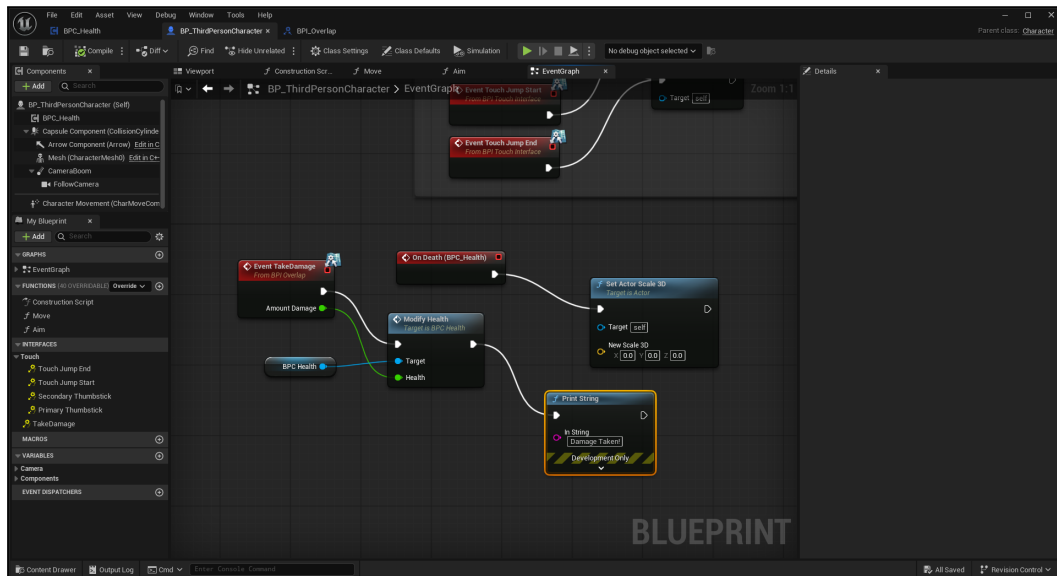
4. **Memeriksa fungsi Interface di panel My Blueprint.** Setelah Blueprint karakter

ter dikompilasi, panel My Blueprint menampilkan kategori **Interfaces** yang memuat fungsi **TakeDamage**. Fungsi ini kini siap diimplementasikan sebagai event penanganan damage di Event Graph karakter.



Gambar 9: Kemunculan fungsi TakeDamage di bawah kategori Interfaces pada panel My Blueprint setelah kompilasi.

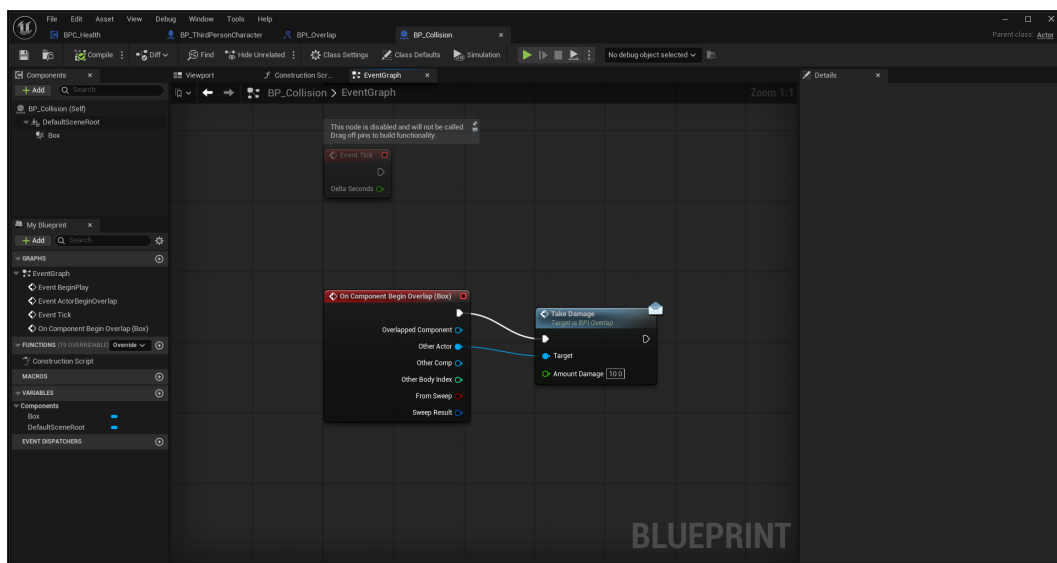
5. **Menyusun logika Event TakeDamage pada karakter.** Fungsi TakeDamage dipasang ke Event Graph dengan klik kanan dan memilih *Implement Event*. Node **Event TakeDamage** menerima nilai **AmountDamage**, lalu menyalurkannya ke pemanggilan fungsi **ModifyHealth** pada komponen BPC_Health. Node **Print String** dengan teks "Damage Taken!" dipasang setelahnya untuk mencetak konfirmasi ke layar selama pengetesan.



Gambar 10: Implementasi Event TakeDamage di BP_ThirdPersonCharacter terhubung ke fungsi ModifyHealth dan Print String.

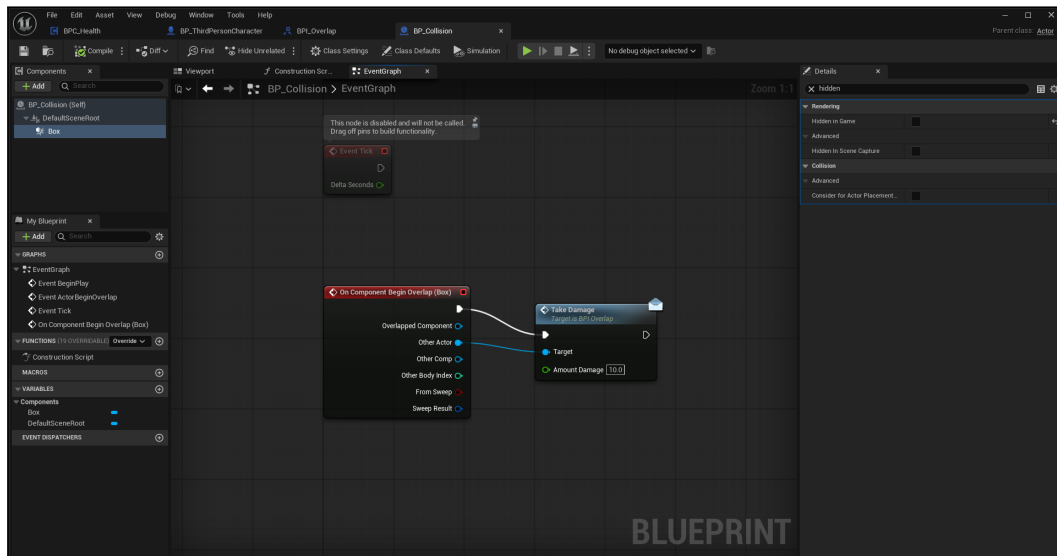
3.1.3 Blueprint Actor Collision dan Instance Editable Parameter (BP_Collision)

1. **Membuat BP_Collision dan merangkai OnComponentBeginOverlap.** Blueprint Actor baru dibuat dengan nama **BP_Collision**. Di dalamnya ditambahkan komponen **Box Collision** dengan opsi *Generate Overlap Events* aktif. Pada Event Graph, event **OnComponentBeginOverlap (Box)** dipasang dan pin **Other Actor** disambungkan ke panggilan pesan interface **TakeDamage (BPI Overlap)**.



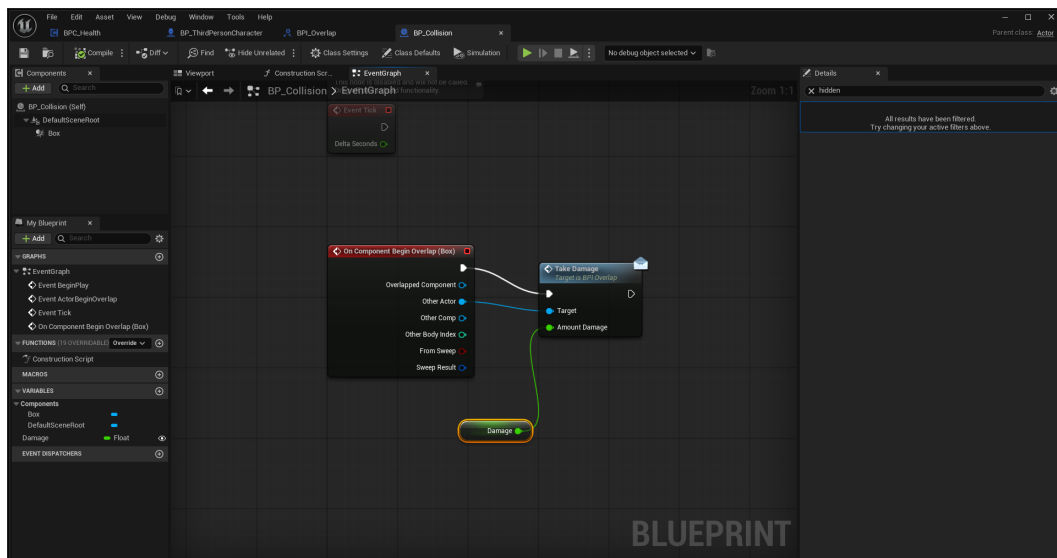
Gambar 11: Event Graph BP_Collision: event OnComponentBeginOverlap memanggil fungsi interface TakeDamage.

2. **Menampilkan visual batas tabrakan di dalam game.** Komponen collision secara baku tidak terlihat saat game dimainkan. Agar batas kotak pemicu kelihatan saat pengujian, komponen **Box** dipilih pada panel Components BP_Collision, lalu pada panel Details kategori **Rendering**, centang pada opsi **Hidden in Game** dimatikan.



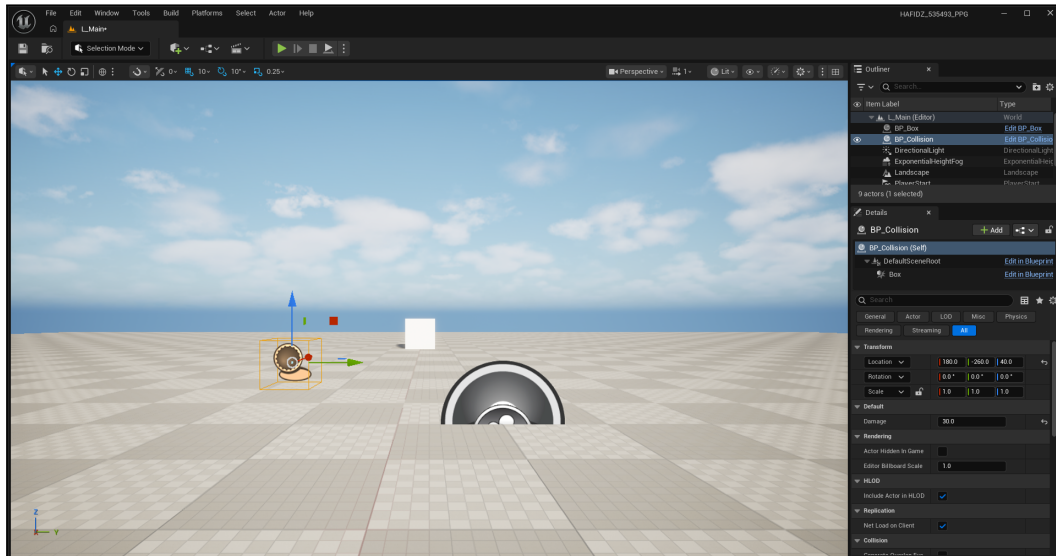
Gambar 12: Penonaktifan opsi Hidden in Game pada Box Collision agar batas pemicu terlihat saat simulasi.

3. **Menambahkan variabel Damage yang bersifat Instance Editable.** Pada panel My Blueprint BP_Collision, variabel float baru bernama **Damage** dibuat dan opsi **Instance Editable** diaktifkan (ikon mata terbuka). Variabel ini disambungkan ke pin input **AmountDamage** pada node panggilan TakeDamage.



Gambar 13: Variabel Damage dengan opsi Instance Editable terhubung ke pin masukan fungsi TakeDamage.

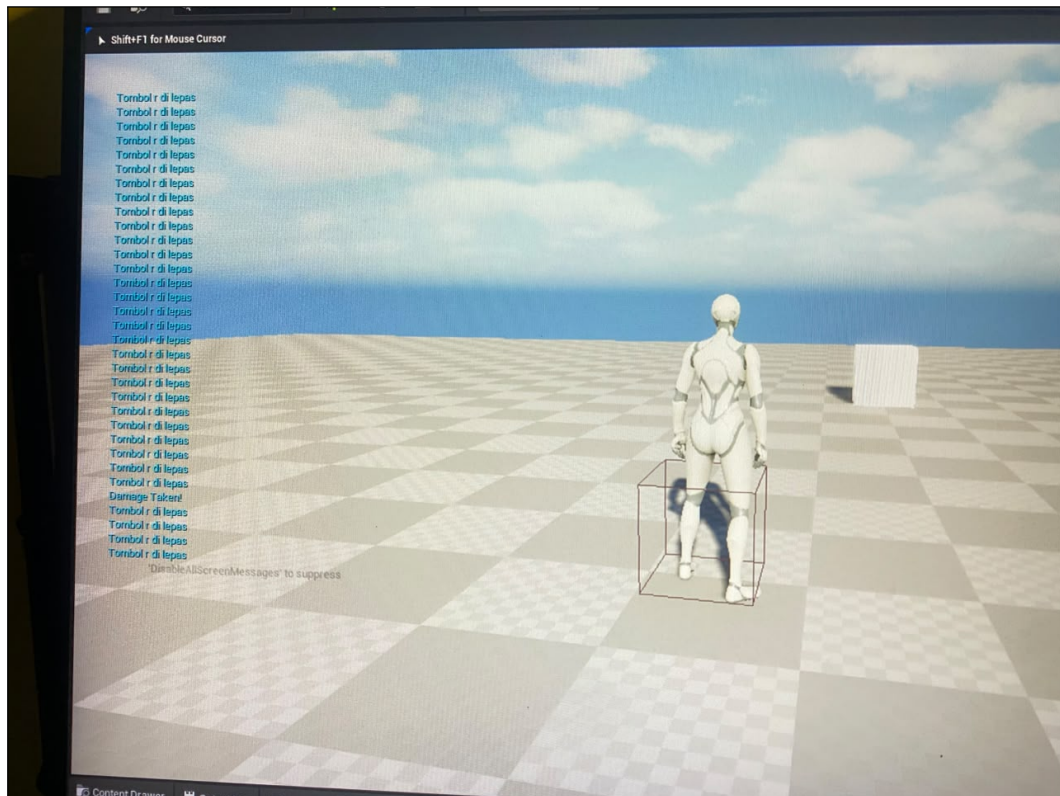
4. **Menempatkan instance BP_Collision dan mengatur nilainya di Level.** Aktor BP_Collision ditarik dari Content Drawer ke arena level. Pada panel Details instance aktor tersebut di level, nilai properti **Damage** diubah menjadi **30.0** langsung melalui editor level tanpa perlu membuka file Blueprint aslinya.



Gambar 14: Pengubahan nilai parameter Damage menjadi 30.0 melalui panel Details instance BP_Collision di level.

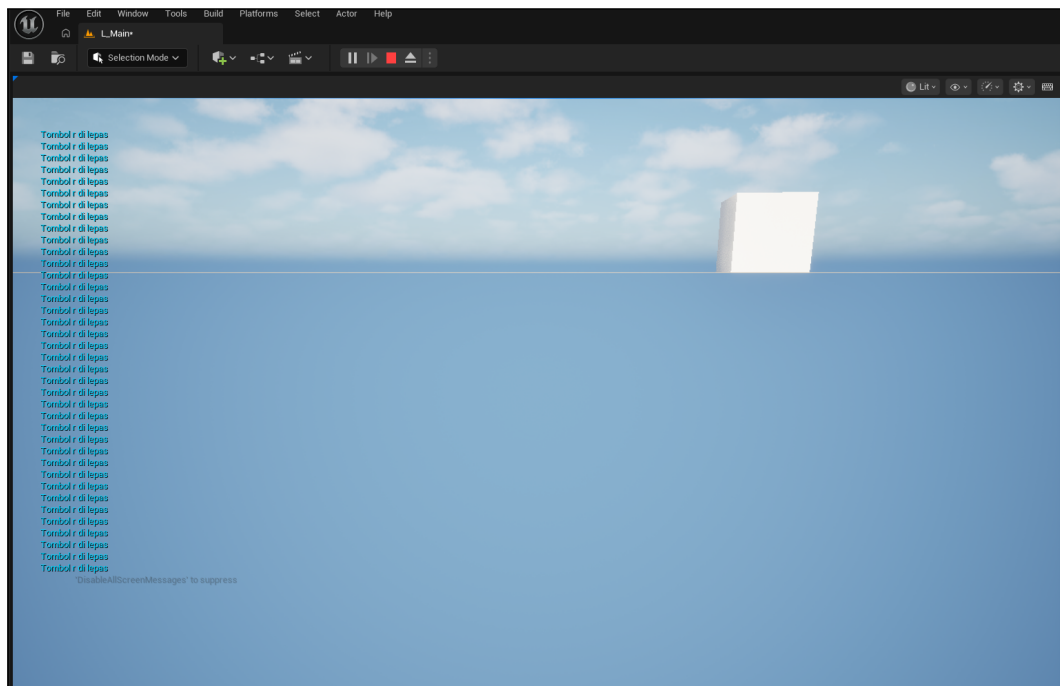
3.2 Hasil Akhir

Rangkaian praktikum diuji langsung dengan menekan tombol Play pada editor level. Karakter Third Person digerakkan melintasi batas kawat pemicu BP_Collision. Kontak tabrakan langsung memicu event overlap yang mengirimkan pesan interface TakeDamage tanpa memerlukan casting kelas secara langsung. Nilai damage sebesar 30.0 diproses oleh komponen BPC_Health untuk mengurangi variabel kesehatan karakter, dan node Print String mencetak teks konfirmasi "Damage Taken!" di pojok kiri atas viewport di sela-sela log status tombol dari pengujian sebelumnya.



Gambar 15: Karakter memasuki area Box Collision, memicu pesan interface TakeDamage dan mencetak teks debug Damage Taken!.

Pengujian dilanjutkan dengan mengarahkan karakter masuk ke dalam volume tabrakan berulang kali sampai akumulasi damage menghabiskan nilai kesehatan. Begitu nilai `CurrentHealth` menyentuh angka nol, fungsi `ModifyHealth` mengeksekusi pemanggilan event dispatcher `OnDeath`. Karakter `BP_ThirdPersonCharacter` merespons event tersebut dengan menyetel skala tubuhnya ke (0.0, 0.0, 0.0). Karakter pun seketika lenyap dari pandangan, menyisakan kubus putih `BP_Box` di latar belakang dan membuktikan seluruh alur logika telah terintegrasi penuh sesuai instruksi Modul 4 [1].



Gambar 16: Karakter kehabisan darah, memicu event dispatcher OnDeath dan melenyapkan karakter dengan skala nol.

Kesimpulan

Dari praktikum Pertemuan 4 ini, saya memahami bahwa pemisahan logika menggunakan Actor Component membuat struktur Blueprint jauh lebih teratur. Komponen kesehatan `BPC_Health` dapat berdiri sendiri dan ditempelkan ke aktor mana pun tanpa bergantung pada susunan internal kelas karakter pemain.

Event Dispatcher `OnDeath` terbukti memisahkan pencatatan data kesehatan dari aksi visual kematian. Komponen kesehatan cukup menyiarkan sinyal, sedangkan karakter pemilik yang menentukan responsnya, seperti mengecilkan skala tubuh hingga nol. Sementara itu, Blueprint Interface `BPI_Overlap` menjadi jalur komunikasi pesan damage yang aman dan bersih tanpa memerlukan casting langsung (Cast to Class).

Terakhir, pemanfaatan Box Collision pada aktor `BP_Collision` dengan variabel Instance Editable memberi kemudahan saat mendesain level. Nilai damage dapat disesuaikan per objek langsung dari panel Details di level tanpa menyentuh file Blueprint asal. Pola desain modular ini menjadi bekal yang rapi untuk membangun mekanisme gameplay yang lebih kompleks pada pertemuan berikutnya.

Daftar Pustaka

- [1] Modul Praktikum Pengembangan Game Pertemuan 4: Interface, Component & Collision, T.A. 2026/2027. Semester Gasal, Universitas Gadjah Mada.
- [2] Epic Games, “Components in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/components-in-unreal-engine>.
- [3] Epic Games, “Event Dispatchers in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/event-dispatchers-in-unreal-engine>.
- [4] Epic Games, “Blueprint Interface in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-interface-in-unreal-engine>.
- [5] Epic Games, “Collision in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-in-unreal-engine>.
- [6] Epic Games, “Collision Response Reference in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/collision-responses-in-unreal-engine>.
- [7] Epic Games, “Blueprint Variables in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprint-variables-in-unreal-engine>.
- [8] Epic Games, “Actors in Unreal Engine”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/actors-in-unreal-engine>.
- [9] Epic Games, “Recommended Asset Naming Conventions in Unreal Engine Projects”, 2026. [Online]. Tersedia: <https://dev.epicgames.com/documentation/en-us/unreal-engine/recommended-asset-naming-conventions-in-unreal-engine-projects>.

- [10] Ali Elzoheiry, “Unreal Engine 5 Beginner Tutorial - Blueprints, Interfaces & Components”, 2023. [Online Video]. Tersedia: <https://www.youtube.com/watch?v=EQfm12D9hwE>.